Generowanie tekstu w stylu Szekspira przy użyciu sieci rekurencyjnych (GRU)

Michał Wilk, Wiktor Zmiendak

Cel ćwiczenia

Celem zadania było stworzenie, wytrenowanie i przetestowanie modelu rekurencyjnej sieci neuronowej (RNN) z wykorzystaniem warstw GRU (Gated Recurrent Unit), zdolnego do generowania tekstu naśladującego styl Williama Szekspira (predykcja znak po znaku).

Przygotowanie i przetwarzanie danych

Jako zbiór uczący wykorzystaliśmy wszystkie dzieła Szekspira.

```
import tensorflow as tf
shakespeare_url = "https://homl.info/shakespeare" # shortcut URL
filepath = tf.keras.utils.get_file("shakespeare.txt", shakespeare_url)
with open(filepath) as f:
  shakespeare_text = f.read()
```

```
Downloading data from https://homl.info/shakespeare
1115394/1115394 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

Wyświetlamy kilka pierwszych wierszy

```
print(shakespeare_text[:100])
```

```
First Citizen:
Before we proceed any further, hear me speak.

All:
Speak, speak.

First Citizen:
You
```

Wektoryzacja tekstu

Surowy tekst poddaliśmy wstępnemu przetwarzaniu:

- Tokenizacji: Podział na znaki (char-level).
- Standaryzacji: Konwersja na małe litery.
- Kodowaniu: Każdy unikalny znak został przypisany do liczby całkowitej.

Odrzuciliśmy tokeny systemowe (0 - padding, 1 - unknown), przesuwając indeksy o 2, aby operować na czystych identyfikatorach znaków.

```
text_vec_layer = tf.keras.layers.TextVectorization(
  split="character",
  standardize="lower"
)
text_vec_layer.adapt([shakespeare_text])
encoded = text_vec_layer([shakespeare_text])[0]
encoded -= 2 # drop tokens 0 (pad) and 1 (unknown), which we will not us
n_tokens = text_vec_layer.vocabulary_size() - 2 # number of distinct cha
dataset_size = len(encoded) # total number of chars = 1,115,394
```

Tworzenie sekwencji (Dataset)

Dane przekształciliśmy w sekwencje uczące za pomocą techniki **okienkowania (windowing)**:

- **Wejście (X):** Sekwencja o długości L.
- **Wyjście (Y):** Ta sama sekwencja przesunięta o jeden "takt" (krok czasowy) w przyszłość.
- **Długość ramki (length):** Ustalona na 100 znaków.
- zastosowaliśmy tasowanie (`shuffle`), grupowanie (`batch_size=32`) oraz wstępne ładowanie (`prefetch`) dla optymalizacji wydajności.

Zbiór danych podzieliliśmy następująco:

- Zbiór treningowy: 90% (pierwszy 1 mln znaków).
- Zbiór walidacyjny: 5%.
- Zbiór testowy: 5%.

```
def to_dataset(sequence, length, shuffle=False, seed=None, batch_size=32
  ds = tf.data.Dataset.from_tensor_slices(sequence)
  ds = ds.window(length + 1, shift=1, drop_remainder=True)
  ds = ds.flat_map(lambda window_ds: window_ds.batch(length + 1))
  if shuffle:
    ds = ds.shuffle(100_000, seed=seed)
```

```
  ds = ds.batch(batch_size)
  return ds.map(lambda window: (window[:, :-1], window[:, 1:])).prefetch
```

```
list(to_dataset(text_vec_layer(["To be"])[0], length=4))
```

```
[(<tf.Tensor: shape=(1, 4), dtype=int64, numpy=array([[ 4,  5,  2,
23]])>,
  <tf.Tensor: shape=(1, 4), dtype=int64, numpy=array([[ 5,  2, 23,
3]])>)]
```

```
length = 100
tf.random.set_seed(42)
train_set = to_dataset(encoded[:1_000_000], length=length, shuffle=True,
seed=42)
valid_set = to_dataset(encoded[1_000_000:1_060_000], length=length)
test_set = to_dataset(encoded[1_060_000:], length=length)
```

Architektura Modelu

Ze względu na rozmiar zbioru danych zastosowaliśmy architekturę zdolną do zapamiętywania dłuższych zależności w tekście.

Model składa się z następujących warstw:

1. **Embedding Layer:**
   - Wejście: Identyfikatory znaków.
   - Wyjście: Wektor o wymiarze 16.
   - Cel: Mapowanie dyskretnych znaków na gęstą reprezentację wektorową.
   - Wymiarowość wyjścia warstwy: *[batch_size, window_length, embedding_dim]*.
2. **GRU Layer (Gated Recurrent Unit):**
   - Liczba jednostek: 128.
   - Parametr `return_sequences=True`: Warstwa zwraca wynik dla każdego kroku czasowego (nie tylko dla ostatniego), co jest kluczowe przy predykcji "znak po znaku" w całej sekwencji.
3. **Dense Layer (Warstwa wyjściowa):**
   - Liczba neuronów: 39 (odpowiada liczbie unikalnych znaków).
   - Funkcja aktywacji: **Softmax**.

- Cel: Zwrócenie rozkładu prawdopodobieństwa dla każdego z 39 możliwych kolejnych znaków.

Proces Uczenia

Model skompilowaliśmy i trenowaliśmy z następującymi hiperparametrami:

- **Funkcja straty:** `sparse_categorical_crossentropy` (odpowiednia dla klasyfikacji wieloklasowej, gdzie etykiety są liczbami całkowitymi).
- **Optymalizator:** `nadam`.
- **Metryka:** `accuracy` (dokładność).
- **Liczba epok:** 10.
- **Callback:** `ModelCheckpoint` – zapisywanie tylko najlepszego modelu na podstawie `val_accuracy`.

```
tf.random.set_seed(42) # extra code ensures reproducibility on CPU
model = tf.keras.Sequential([
  tf.keras.layers.Embedding(input_dim=n_tokens, output_dim=16),
  tf.keras.layers.GRU(128, return_sequences=True),
  tf.keras.layers.Dense(n_tokens, activation="softmax")
])
model.compile(
  loss="sparse_categorical_crossentropy",
  optimizer="nadam",
  metrics=["accuracy"]
)
model_ckpt = tf.keras.callbacks.ModelCheckpoint(
  "my_shakespeare_model.keras",
  monitor="val_accuracy",
  save_best_only=True
)
history = model.fit(
  train_set,
  validation_data=valid_set,
  epochs=10,
  callbacks=[model_ckpt]
)
```

```
Epoch 1/10
  31247/Unknown 3432s 109ms/step - accuracy: 0.5481 - loss: 1.4946/usr/lo
  self._interrupted_warning()
31247/31247 ━━━━━━━━━━━━━━━━━━━━ 3494s 111ms/step - accuracy: 0.5481 - lo
Epoch 2/10
31247/31247 ━━━━━━━━━━━━━━━━━━━━ 3483s 110ms/step - accuracy: 0.5973 - lo
Epoch 3/10
```

```
31247/31247 ━━━━━━━━━━━━━━━━━━━━ 3529s 112ms/step - accuracy: 0.6018 - lo
Epoch 4/10
31247/31247 ━━━━━━━━━━━━━━━━━━━━ 3562s 112ms/step - accuracy: 0.6043 - lo
Epoch 5/10
31247/31247 ━━━━━━━━━━━━━━━━━━━━ 3541s 112ms/step - accuracy: 0.6055 - lo
Epoch 6/10
31247/31247 ━━━━━━━━━━━━━━━━━━━━ 3522s 112ms/step - accuracy: 0.6063 - lo
Epoch 7/10
31247/31247 ━━━━━━━━━━━━━━━━━━━━ 3563s 112ms/step - accuracy: 0.6030 - lo
Epoch 8/10
31247/31247 ━━━━━━━━━━━━━━━━━━━━ 3520s 112ms/step - accuracy: 0.6077 - lo
Epoch 9/10
31247/31247 ━━━━━━━━━━━━━━━━━━━━ 3562s 113ms/step - accuracy: 0.6078 - lo
Epoch 10/10
31247/31247 ━━━━━━━━━━━━━━━━━━━━ 3540s 113ms/step - accuracy: 0.6086 - lo
```

Implementacja modelu końcowego (Inference)

Po wytrenowaniu, surowy model (operujący na liczbach) został opakowany w strukturę "end-to-end", aby umożliwić przyjmowanie tekstowych danych wejściowych:

1. **Warstwa wejściowa:** `TextVectorization` (zamiana tekstu na indeksy).
2. **Warstwa Lambda:** Korekta indeksów (odejmowanie 2).
3. **Model właściwy:** Wytrenowany model GRU.

```
shakespeare_model = tf.keras.Sequential([
  text_vec_layer,
  tf.keras.layers.Lambda(lambda X: X - 2), # no <PAD> or <UNK> tokens
  model
])
```

Przykłady predykcji

Dla wejścia *"To be or not to b"*:

1. Model zwraca prawdopodobieństwa dla ostatniego znaku.
2. Wyznaczany jest indeks o najwyższym prawdopodobieństwie (`argmax`).
3. Indeks zamieniany jest z powrotem na znak w tym przypadku "e".

```
y_proba = shakespeare_model.predict(tf.constant(["To be or not to b"], d
y_pred = tf.argmax(y_proba) # choose the most probable character ID
text_vec_layer.get_vocabulary()[y_pred + 2]
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 960ms/step
np.str_('e')
```

Widzimy też że predykcja działa poprawnie gdy początek sekwencji jest skrócony.

```
y_proba = shakespeare_model.predict(tf.constant(["not to b"], dtype=tf.s
y_pred = tf.argmax(y_proba) # choose the most probable character ID
text_vec_layer.get_vocabulary()[y_pred + 2]
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 56ms/step
np.str_('e')
```

Natomiast dla jednego z fragmentów w którym prawidłową predykcją powinno być `k` otrzymujemy `r` ze względu na wyższe prawdopodobieństwo wystąpienia tej litery, zapewne przez nazwisko autora utworów na których model był trenowany.

```
y_proba = shakespeare_model.predict(tf.constant(["Speak ,spea"], dtype=t
y_pred = tf.argmax(y_proba) # choose the most probable character ID
text_vec_layer.get_vocabulary()[y_pred + 2]
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 51ms/step
np.str_('r')
```